Big Data Analytics Assignment

ShopSphere: A Big Data Customer-Intelligence Platform for E-Commerce

Hadoop ecosystem + Cassandra solution for personalization, demand forecasting and fraud detection

1. Problem Statement

ShopSphere is an online marketplace with 20 million registered users, 2 million daily visitors and 500,000 orders per day. Its data is spread across separate systems (order database, web logs, app events, social media, warehouse systems). A traditional RDBMS data warehouse cannot ingest clickstream data at this rate, cannot analyze free text such as reviews, and delivers reports a day late. As a result the company suffers from generic recommendations, stock-outs and overstocking, cart abandonment, and undetected fraud.

> **Goal:** build a scalable platform that collects all customer and operational data, stores it cheaply, analyzes it in batch and near real time, and delivers insights to business users on web and mobile dashboards.

Characteristics of the data (5 Vs)

V	In ShopSphere
Volume	About 2 TB of new data per day (clicks, orders, logs, reviews). Roughly 700 TB per year, growing as traffic grows.
Velocity	Around 50,000 click events per second at peak (sale days). Fraud checks and recommendations must respond in seconds.
Variety	Relational tables, JSON/CSV logs, free-text reviews, social posts, product images and sensor readings.
Veracity	Bot traffic, fake reviews, duplicate events, missing fields, inconsistent product IDs and time-zone differences make raw data unreliable and need cleansing.
Value	Better recommendations raise conversion, accurate forecasts cut inventory cost, and fraud detection reduces losses.

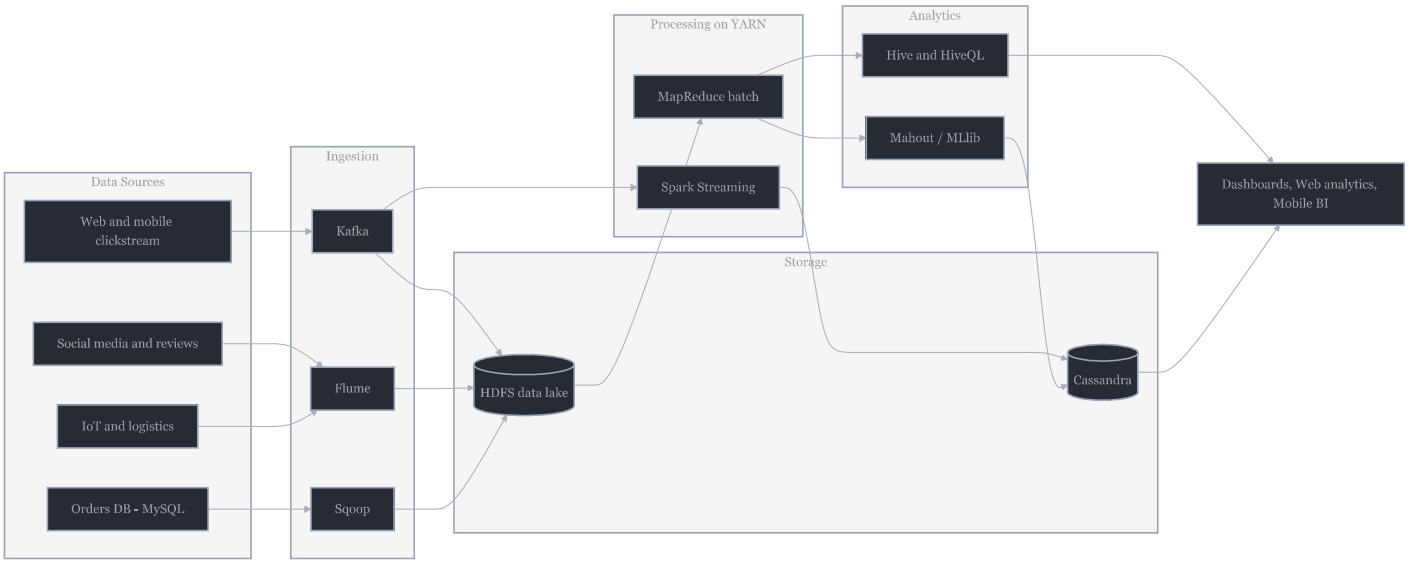
	detection reduces losses.

2. Data Types and Sources

Category	Data	Source
Structured	Orders, payments, customers, product catalogue, inventory, returns	MySQL/Oracle OLTP systems, ERP
Semi-structured	Clickstream and app events (JSON), server logs, API responses, email campaign logs	Website, Android/iOS apps, web servers
Unstructured	Product reviews, support chats, social media posts, product images, call recordings	Review module, Twitter/Instagram feeds, helpdesk
Machine data	Warehouse scanner, delivery vehicle GPS, temperature sensors	IoT devices, logistics partners

3. System Architecture (Hadoop Ecosystem)

The design is a layered (Lambda-style) architecture: a batch layer for large historical analysis and a speed layer for near-real-time events, with a serving layer for fast lookups.



4. Data Flow Diagram



5. Technology Stack

Layer	Technology	Purpose
Ingestion	Apache Kafka, Flume, Sqoop	Streaming events, log/social collection, bulk RDBMS import
Storage	HDFS (Parquet/ORC files), Apache Cassandra	Data lake; low-latency serving store
Resource mgmt	YARN, ZooKeeper, Oozie	Cluster scheduling, coordination, workflow automation
Processing	MapReduce, Spark Streaming	Batch ETL and aggregation; real-time scoring
Querying	Hive / HiveQL, Pig	SQL-style analysis and data preparation
Machine learning	Mahout, Spark MLlib	Collaborative filtering, forecasting, classification
Cloud	AWS (EMR, S3, EC2) or Azure HDInsight	Elastic, pay-as-you-go cluster
Visualization	Tableau/Superset, Google Analytics, mobile BI app	Dashboards and web analytics
Security	Kerberos, Ranger, TLS	Authentication, access control, encryption

6. Justification of Hadoop and NoSQL Technologies

Why Hadoop

- **Scalability:** commodity servers can be added horizontally as data grows from terabytes to petabytes.
- **Cost:** open source and runs on cheap hardware, much cheaper than a proprietary warehouse for raw historical data.
- **Fault tolerance:** HDFS replicates each block (factor 3), so a node failure does not lose data or stop jobs.
- **Schema-on-read and variety:** structured, semi-structured and unstructured data can all be stored first and interpreted later.
- **Data locality:** computation is moved to where the data lives, reducing network traffic

Data locality: computation is moved to where the data lives, reducing network traffic.

Why Cassandra (column-family NoSQL)

The platform needs to serve millisecond lookups such as "top 10 recommendations for user 8841" and to absorb a constant stream of writes (session state, event counters, fraud scores). Options compared:

Option	Assessment for this use case
Cassandra	**Selected.** Masterless ring with no single point of failure, very high write throughput, linear scaling, tunable consistency, multi-datacenter replication, and built-in TTL for expiring session data. Fits time-series and user-profile queries modelled by partition key.
HBase	Good fit on top of HDFS and strongly consistent, but has a master/region-server design that is more complex to operate and is less suited to multi-region availability.
Document store (MongoDB)	Flexible JSON, but heavy sustained writes at this scale are harder to shard and tune.
Key-value (Redis)	Fastest, but memory-bound and costly for large data; useful only as an optional cache.
Graph (Neo4j)	Excellent for relationship analysis, but does not scale horizontally as easily. Kept as a future enhancement for fraud rings.

Sample Cassandra table: `user_recs(user_id, rank, product_id, score) PRIMARY KEY (user_id, rank)` so one read fetches a user's recommendations.

7. Storage in HDFS and Processing with MapReduce

HDFS storage design

- **Block size** 128 MB, **replication** factor 3, NameNode with standby for high availability.
- **Zones:** `/data/raw` (as ingested, never modified), `/data/curated` (cleaned, Parquet/ORC with compression), `/data/analytics` (aggregates).
- **Partitioning:** `/data/curated/clicks/year=2026/month=10/day=03/` so queries scan only needed days.

- Small log files are merged into large files, since HDFS handles few big files better than many small ones.

MapReduce job: top-selling products and revenue per category

Input: order-item records `order_id, product_id, category, qty, price`.

```
# Mapper: emit (category, revenue) for each valid order line
def map(line):
    f = line.split(",")
    if len(f) == 5 and f[3].isdigit():          # cleansing: skip malformed rows
        emit(f[2], int(f[3]) * float(f[4]))

# Reducer: sum revenue for each category
def reduce(category, revenues):
    emit(category, sum(revenues))
```

Execution: input splits are processed by parallel mappers on the nodes holding the blocks, then shuffle and sort groups values by category, and reducers write totals back to HDFS. A combiner is added to pre-sum on each mapper and cut network traffic. Other MapReduce jobs: log cleansing and bot removal, co-purchase counting for "customers also bought", and sessionization of clicks.

8. Querying with Hive / HiveQL

Hive maps SQL-like tables onto HDFS files so analysts can query petabytes without writing Java; queries compile to MapReduce/Tez jobs.

```
CREATE EXTERNAL TABLE orders (
  order_id BIGINT, user_id BIGINT, product_id INT,
  category STRING, qty INT, price DOUBLE)
PARTITIONED BY (order_date STRING)
STORED AS ORC LOCATION '/data/curated/orders';

-- Monthly revenue by category
SELECT category, SUM(qty*price) AS revenue
FROM orders WHERE order_date LIKE '2026-09%'
GROUP BY category ORDER BY revenue DESC;

-- Cart abandonment rate
SELECT COUNT(DISTINCT CASE WHEN event='add_to_cart' THEN session_id END) AS carts,
       COUNT(DISTINCT CASE WHEN event='purchase' THEN session_id END) AS purchases
FROM clickstream WHERE dt = '2026-10-02';
```

```
-- Top 10 customers by lifetime value
SELECT user_id, SUM(qty*price) AS ltv FROM orders
GROUP BY user_id ORDER BY ltv DESC LIMIT 10;
```

Partitioning, ORC format and bucketing keep these queries fast. Results feed dashboards through JDBC/ODBC.

9. Role of Cloud Computing, Web Analytics and Mobile BI

- **Cloud computing:** hosting the cluster on AWS EMR with S3 storage lets ShopSphere scale up for festival sales and shut clusters down afterwards, paying only for usage, avoiding upfront hardware cost and gaining managed backups and disaster recovery.
- **Web analytics:** clickstream is analyzed for traffic sources, bounce rate, funnel drop-off, heat maps and A/B tests, which tells the product team which pages and campaigns work.
- **Mobile BI:** managers, category heads and delivery supervisors receive KPI dashboards and alerts (sales vs target, stock-outs, fraud spikes) on their phones for decisions on the go.

10. Expected Outcomes, Benefits and Challenges

Business insights

- Customer segments (price-sensitive, loyal, at risk of churn) and lifetime value.
- Product affinity and personalized recommendations.
- Demand forecasts by product and region, with seasonal patterns.
- Sentiment from reviews and social posts about products and delivery.
- Fraud patterns such as unusual order bursts, repeated failed payments, and fake reviews.

Benefits

Area	Expected result (target)
Sales	8-12% higher conversion from personalization
Inventory	15-20% lower stock-out and overstock cost
Fraud	Faster detection, reducing losses by 30%+

Reporting	From next-day reports to minutes
Cost	Lower storage cost than a traditional warehouse

These figures are planning targets, not measured results.

Challenges and mitigation

Challenge	Mitigation
Data quality (veracity)	Cleansing jobs, validation rules, bot filters, data governance
Privacy and compliance (India DPDP Act, GDPR)	Encryption, masking and anonymization of personal data, consent tracking, access control
Skills and operations	Training, managed cloud services, monitoring
Small files and slow MapReduce	File compaction, Tez/Spark for interactive work
Integration of legacy systems	Sqoop connectors, phased migration
Cost control in cloud	Auto-scaling, spot instances, storage tiering

Future enhancements

- Replace most MapReduce jobs with Apache Spark for faster in-memory processing.
- Add a graph database to detect organized fraud rings and refine social recommendations.
- Deep learning for product image search and review sentiment analysis.
- Voice and chatbot assistants using NLP on support data.
- Edge/IoT analytics for smart warehouses and delivery route optimization.
- Data catalog, lineage and automated governance.

Conclusion

The proposed platform uses Kafka, Flume and Sqoop to ingest data, HDFS as a low-cost data lake, MapReduce and Spark for processing, Hive for SQL analytics and Cassandra for real-time serving, all hosted on the cloud. It addresses all five Vs of ShopSphere's data and turns raw events into recommendations, forecasts and fraud alerts that directly improve revenue and cost.